

C17 — Launch, Rollout and Execution Decisions

1. What interviewers test

"You are launching this next month. Walk me through it." "The experiment shows a lift but also slower pages. Do you launch?" "Two hours after launch the error rate is up. What now?" "A defect turns up two weeks before manufacturing locks." These questions ask for judgment at the point where a team commits to the world: what has to be true before you ship, how you expose users to the change, how you decide when results are mixed, and how you act when things go wrong. The interviewer is looking for a PM who treats a launch as a managed risk with criteria set in advance, not as a date.

Where it shows up. Inferred from the round structures candidates commonly report: launch, rollout and rollback scenarios at Google, Meta, Microsoft, Amazon, Apple, Netflix, Stripe and others, often as the closing execution question after a design or prioritization answer, and as follow-ups to experiment readouts [unverified: candidate-reported]. For hardware companies a distinct form appears, where the product cannot be updated after it ships. On the job it is the work of the days before and after a release: readiness reviews, staged ramps, monitoring, decisions under pressure.

What is scored. Whether your launch thinking is criteria-based, staged and honest about risk Inferred. Interviewers listen for a definition of readiness that goes beyond "the code is done"; a rollout that limits the blast radius and is sized to detect harm; pre-agreed rollback triggers with thresholds that respect noise; a clear way to decide when results are mixed; an understanding that hardware changes the rules; and a post-launch plan to measure and learn.

Six observable behaviors:

1. You define launch readiness with criteria and owners, and name the go or no-go decider.
2. You choose a rollout strategy (flags, canary, staged ramp) and say how each stage is sized and what advances it.
3. You set rollback triggers in advance, with thresholds and windows that account for noise.
4. You decide launch or hold from experiment results using a pre-set rule, including partial launches.
5. You recognize irreversibility (hardware, public commitments) and adapt evidence, staging and supply planning.
6. You plan the post-launch review: what you will measure, when, and what would make you iterate, scale or stop.

The failure this prevents. Six patterns end launch answers: "we launch when the code is done," a 100% release on a Friday with no way back, rollback decided in the middle of the incident by whoever is loudest, a ramp too small to detect the harm you worry about, treating hardware like software, and no plan to learn after launch.

2. The concept from first principles

C17.1 Defining launch readiness

A launch is a state of readiness crossed on purpose, not a date. A team is ready when a short list of criteria is met, each with an owner, and a named person decides go or no-go. FW-C17-01 is the gate.

Category	The question	Example criterion
Product	Does it meet its acceptance criteria and quality bar (C16.9)?	No open S1; S2 only with workaround and owner
Reliability	Will it hold up, and can we undo it?	Load tested to 2× expected peak; rollback rehearsed
Data	Can we tell whether it worked?	Events logged and checked; dashboards live; baseline recorded
Support	Can people get help?	Scripts and training done; escalation route named
Communication	Do those affected know?	Internal note, customer message, status page ready
Legal, privacy, security	Have constraints been met?	Sign-offs recorded; open risks accepted by a named owner
Accessibility and localization	Does it work for everyone intended?	Accessibility pass done (C10.6); languages complete
Rollout plan	How will exposure grow?	Stages, triggers and owners agreed

The **go or no-go** decision belongs to one named person (often the PM with the engineering lead), informed by the checklist, with dissent recorded. A meeting where "everyone agrees" and nobody owns the decision is a risk in itself. Readiness is also graded by stakes: a copy change needs a light gate, a payments change needs the full one (C02's decision classifier).

When not to hold a heavy gate. For small, reversible changes behind a flag with automatic monitoring, a light gate and a fast rollback beat a long review. Process should scale with the cost of being wrong.

Counterexample. A team announces "launch is Friday" in week one and treats the date as the goal. Testing finds problems on Thursday; there is no rollback rehearsal; support learns of the change from customers. The date was met and the launch failed. A team that wrote the criteria in week one, assigned owners and had a go or no-go meeting on Wednesday would have found the gaps with time to act, and could have chosen a smaller launch on Friday.

Hidden risks. Two. *Checklist theater*: every box ticked and nobody asked whether the criteria were the right ones. *Deadline gravity*: the closer the date, the more the criteria bend. Write the criteria early and agree that only the decider may waive one, in writing.

C17.2 Rollout strategies: staged releases, feature flags, canaries

The safest way to launch is to expose a change to a few users first and widen only when it behaves. Four ideas make that possible.

A **feature flag** is a switch that turns functionality on or off for chosen users without shipping a new release. A **staged rollout** is releasing a change to progressively larger shares of users while monitoring for problems. A **canary release** is a first release to a small subset of users or servers that acts as an early warning for the rest; the name comes from the canary in the coal mine. **Blast radius** is how many users or systems a failure would affect. Staging and flags exist to shrink the blast radius.

A rollout ladder (FW-C17-02). A typical ladder: internal use (employees), a canary (1% of users), 10%, 25% or 50%, then 100%, with a minimum dwell time at each stage and a set of conditions to advance: no rollback trigger hit, guardrails within margins, no unexplained anomalies. Each stage has an owner who watches the dashboards, and a defined way back (turn the flag off).

Size the stages to detect the harm you fear. A stage that is too small cannot see a modest problem. Take an error rate of 1% (invented). To detect a rise of 0.5 points (to 1.5%) with 80% power at the 5% level, with the rest of the users as the comparison, a stage needs about 4,700 users. To detect a rise of 0.3 points it needs about 11,300; to detect 0.1 points, about 93,400. On a base of 1,000,000 users, a 1% canary (10,000 users) can reliably see a rise of about 0.33 points or more; seeing 0.1 points needs about 9% exposure. So the 1% canary protects against severe failures, and the move to 10% is what protects against subtle ones. Choose stage sizes and dwell times from the smallest harm you are unwilling to accept, using the same arithmetic as in C12 and C14.

Dwell time. Some problems appear only with time or scale: a memory leak after hours, a weekly cycle, a rare path. Hold each stage long enough to cover the cycle that matters (at least a business day, and a week for weekly patterns), and watch the lagging indicators (support contacts, refunds) as well as the fast ones.

Other forms. *Dark launch*: run the new code path in production without showing results, to test load and correctness. *Geographic staging*: start in one market. *Opt-in beta*: users choose, which gives goodwill and a biased sample. *Percentage ramps by cohort*: new users first. *Kill switch*: a flag that turns off a risky component fast.

Why it works. Staging trades a little speed for a much smaller worst case. **When not to stage.** When the change cannot be partially released (a database migration, a legal change on a date) or when staging itself adds risk; then test harder before, and plan the way back.

C17.3 Hardware and device launches: why irreversibility changes rollout, rollback and supply planning

Software can be changed after release; most hardware cannot. This single fact changes the logic of a launch.

What is different.

- **No simple rollback.** A shipped device is in a customer's hands. The nearest equivalents are a software or firmware update (if the device can receive one and the defect is fixable that way), a repair or replacement program, or a recall, each slow and expensive.
- **Lock dates.** Tooling, component orders and manufacturing slots are committed months ahead. After a lock date, changing a physical part means a delay measured in weeks and a cost measured in millions.
- **Supply planning.** The first production run is a bet on demand: too few units means lost sales and a launch with shortages; too many means inventory risk. Lead times for components and second runs are long.
- **Certification.** Many devices need safety and regulatory approvals before sale, which fix the design [V: FCC equipment authorization, 47 CFR Part 2 and section 2.1043 on firmware (permissive changes); US rules for radio devices only; safety approvals and non-US regimes not checked].
- **The update path is part of the product.** A fix-later plan only works if the device can receive the update. If the defect breaks the connection or the first-boot step that updates need, then "we'll fix it in software" is not a plan.

How the approach changes (FW-C17-03). Classify each decision by reversibility: reversible (a flag), hard to reverse (a public price), irreversible (a manufactured part). Demand more evidence for the irreversible: more testing before the lock (reliability, abuse, environmental, field trials with real users), explicit gating of the lock on test results, staged availability by region or channel, and a validated fix-later path. Quantify the cost of each option, including the cost of a defect in the field (returns, support, replacement, reputation) against the cost of a delay.

Decision under irreversibility. When a defect appears before the lock, the options are usually: delay the lock, lock on time with a validated update path for the fix, or lock on time and accept the defect. Worked example 3 sets out the arithmetic and a gating approach: validate the fix path early enough that you can still delay if it fails.

Safety is different. A defect that can hurt people is not a tradeoff against revenue. The bar is much higher, and the case in section 10 shows the cost of getting it wrong.

Why it works. It shifts effort to before the lock, where changes are cheap, and keeps an escape route. **When not to over-apply it.** To software-only products, where staging and rollback are available and cheaper than extra pre-launch testing.

C17.4 Launch or no launch, given experiment results

Experiment results rarely come out clean. Use the pre-set decision rule from C14.5 (FW-C14-05), and extend it for launch decisions with partial options (FW-C17-04). The possible actions are: *launch to everyone*, *launch in stages with a long-term holdout*, *extend the test* (the interval is too wide), *fix and retest* (a guardrail breached for a reason you can address), or *do not launch*.

Read results in the order of C14.8: validity first (sample ratio, novelty), guardrails next, then the primary metric, then secondary metrics and segments, then the time horizon. Then ask launch-specific questions: what is the cost of being wrong in each direction (a false launch against a missed launch), is the decision reversible (it often is, behind a flag), does the effect last (novelty), and is a long-term holdout in place to check.

Take an example (invented). The primary metric, purchases, shows +0.4 points with an interval of +0.1 to +0.7, against a smallest worthwhile effect of +0.3. The latency guardrail shows +30 ms with an interval of +10 to +50 against a margin of +50 ms: within the margin, just. The support-contacts guardrail shows +2% with an interval of -1% to +5% against a margin of +3%: the upper end breaches. Under the strict rule from C14.5 the answer is not to launch as is: the primary interval's lower end is below the threshold and a guardrail breaches. A sensible plan: investigate the support contacts (what are they about?), extend the test to narrow the interval (four times the data halves it), and if the cause is a confusing label, fix and retest. The value at stake gives urgency: +0.4 points on 5,000,000 visitors a month at \$10 a purchase is about \$200,000 a month, so each month of delay has a price, and a staged launch with the guardrails watched is a reasonable alternative if the cause is understood and the rollback is quick. The key is to decide on the rule and the cost of delay, not on impatience.

Why it works. The decision follows a rule written before the data, and partial options avoid a false choice between all and nothing. **When not to wait for certainty.** When the change is reversible, the downside is bounded by staging, and the cost of delay is large.

C17.5 Rollback triggers and incident response at PM level

A **rollback** is reverting a release to its previous state. The most important decision about rollback is made before launch: the conditions under which you will do it. A **rollback trigger** is a pre-agreed condition that, when met, causes a release to be reverted or paused. FW-C17-05 is a sheet: for each trigger, the metric, the threshold, the window, the owner and the action.

Thresholds must respect noise. A threshold inside the normal variation fires on chance; one far outside it fires too late. Use the noise band (C13, C12). Take a payment success rate with a baseline of 96% and about 1,000 attempts per half hour (invented). The standard error of the rate over half an hour is $\sqrt{(0.96 \times 0.04 \div 1,000)} = 0.62$ points, so a drop of 1.0 point is only 1.6 standard errors: it will happen by chance often. Over two hours, with 4,000 attempts, the standard error is 0.31 points, and a 1.0-point drop is 3.2 standard errors: a reliable signal. So the trigger might read: "Payment success below 95.0% over a rolling two hours, or below 92% over any 15 minutes, which is a large drop that is visible even in a small window." Pair a sensitive-but-slow trigger with a fast-but-coarse one.

Who decides. The trigger names the owner and the action, so that no one has to argue at 2 a.m. Typically the on-call engineer can roll back immediately on a hard trigger; the PM and engineering lead decide on judgment calls.

Incident response at PM level (FW-C17-06). In an incident, engineers work the technical problem under an incident lead. The PM's job is different and no less important:

1. *Confirm the impact:* who is affected, how many, how badly (C13 step zero).
2. *Contain first:* turn off the flag or roll back if a trigger is hit; diagnose after.
3. *Communicate:* internal status at a fixed cadence; the customer-facing message; support told what to say (the five-line status from C13.8).
4. *Decide on scope and tradeoffs* the team brings to you: roll back versus fix forward, partial disable, customer remediation.
5. *Protect the team:* one voice to leaders, fewer interruptions for the people fixing.
6. *Record:* a timeline as it happens.

A **postmortem** is a blameless review after an incident, documenting the timeline, causes, what went well and badly, and actions with owners. Blameless means the question is what in the system allowed this, not who is at fault; people then report problems early. The PM writes or co-owns the product-impact and decision sections, and makes sure the actions get done.

Why it works. Triggers made in calm are better than decisions made in crisis, and a PM who handles communication frees engineers to fix. **When not to roll back.** When the cost of rolling back exceeds the harm (a data migration that cannot be reversed without loss), or when a fast fix forward is clearly safer; say so and decide deliberately.

C17.6 Delivery risk: dependencies, estimates, scope cuts

Launches slip because of things outside the team's control: other teams, vendors, approvals, hardware lead times. FW-C17-07 is a map.

List dependencies and rate them. For each: what is needed, from whom, by when, the probability it arrives on time, the impact if late, and the mitigation. **Risk compounds.** If a launch needs four independent dependencies and each is 90% likely to be on time, the chance that all four are on time is $0.9^4 = 66\%$. With five at 90%, it is 59%. With six at 92%, 61%. A plan that treats each as "probably fine" is a plan that has a one-in-three chance of

slipping. Reduce the count of dependencies on the critical path, raise the probability of the weakest, add buffer where they cluster, and start the longest-lead items first. If two of four are raised to 97% by early engagement, the chance rises to $0.9^2 \times 0.97^2 = 76\%$.

Estimates. Use ranges and the outside view (C16.4), re-estimated at checkpoints. **Scope cuts.** Decide in advance the order in which you would cut if the date is threatened: first nice-to-haves with high effort and low value (C15's value-per-effort), never the launch criteria themselves (quality bar, safety, accessibility). A staged launch is itself a scope cut: launch to fewer users, or with fewer features, on the date, and widen later.

Track and escalate early. Review the map weekly; a dependency at risk is raised the week you see it, with options, not the week it fails.

Why it works. Seeing the compounding turns vague worry into a number, and an ordered cut list turns panic into a plan. **When not to map everything.** For a small change with one or two dependencies, a short note is enough.

C17.7 Pre-launch checklist

The checklist is the gate (C17.1) made concrete. A compact version, to adapt:

1. **Product and quality:** acceptance criteria met; defects triaged by severity; no open S1; accessibility pass done.
2. **Reliability:** load test done; monitoring and alerts live; rollback rehearsed; flag off-switch tested.
3. **Data:** instrumentation verified in production; dashboards and baselines ready; experiment or holdout configured; success criteria written.
4. **Rollout:** stages, dwell times, ramp criteria and rollback triggers agreed; owners named; on-call arranged (including the PM for decisions).
5. **People:** support trained and scripts ready; sales and success informed; internal and external messages drafted.
6. **Constraints:** legal, privacy, security and policy sign-offs recorded; open risks accepted in writing.
7. **Decision:** go or no-go meeting held; decider named; dissent recorded.
8. **After:** post-launch review dates set (day 1, week 1, week 4); owners for follow-up.

C17.8 Post-launch measurement and iteration

Launch is a beginning. Plan the review before you ship (FW-C17-08). At day one, check health: errors, performance, support contacts, rollback triggers. At week one, check the primary metric against the target, guardrails and early cohorts, and look for novelty (C14.4): an early peak is not the steady state. At week four or later, check retention and the long-term holdout, and decide: *iterate* (the direction is right, details to improve), *scale* (invest more), or *stop* (it did not work, withdraw it and say why). Compare with the success criteria you wrote before launch, not with a new story afterward.

If a metric regresses after launch, use the diagnosis tools of C13: confirm it is real, locate it, compare with the launch timeline, and decide on rollback or fix. Record what you learned in a short launch review so that the next team begins with it. Include what surprised you; it is where the learning is.

C17.9 Interview forms

Section 4 sets out the answer architecture and the types: walk me through a launch; the metric regressed after launch; launch or hold given results; a hardware defect before a lock date; engineering says three times longer; define success for X; and an incident at 2 a.m.

C17.10 Worked examples

Section 5 gives five worked examples: sizing a rollout ladder, a launch or hold decision with mixed results, a hardware defect before a manufacturing lock, rollback triggers that respect noise, and compounding delivery risk. Section 11 adds seven questions with model answers, including an Apple hardware question and an Amazon launch question.

3. Frameworks

Eight frameworks, set out below. The experiment decision-rule table (FW-C14-05), the conflicting-metrics readout (FW-C14-07), the status-message template (FW-C13-07), the estimate conversation (FW-C16-04), the quality bar and severity rules (C16.9) and the decision classifier (FW-C02-04) are reused, not repeated.

FW-C17-01 · The launch readiness gate · Original

- **Inputs:** a planned launch and its stakes.
- **Outputs:** a checklist across eight categories (product, reliability, data, support, communication, legal and security, accessibility and localization, rollout) with an owner for each, and a named go or no-go decider.
- **Assumptions:** criteria are written before the date pressure builds.
- **Tradeoffs:** a gate costs time and prevents surprises; scale it to the stakes.
- **Failure modes:** a date as the goal; criteria waived silently; no named decider.
- **Worked example:** the gate in CS-C17-01 and the checklist in C17.7.

FW-C17-02 · The rollout ladder · Original

- **Inputs:** the change, the user base, the smallest harm you will not accept.
- **Outputs:** stages with exposure share, dwell time, advance conditions and an owner; stage sizes chosen to detect the target harm.
- **Assumptions:** a flag or release mechanism allows partial exposure; the control is the remaining users.
- **Tradeoffs:** more stages are safer and slower.

- **Failure modes:** a canary too small to detect the harm; too short a dwell time; no way back.
- **Worked example:** Worked example 1.

Stage	Exposure	Dwell	Advance if	Owner
Internal	Staff	1 day	No S1; instrumentation correct	Eng lead
Canary	1%	1 business day	No trigger hit; guardrails within margin	On-call
Early	10%	2 days	Same; subtle harms ruled out at the planned size	PM and eng lead
Broad	50%	3 days	Same; support contacts normal	PM
Full	100%	—	Long-term holdout kept	PM

FW-C17-03 · The irreversibility check · Original

- **Inputs:** a decision and the options.
- **Outputs:** a reversibility class (reversible, hard to reverse, irreversible), the evidence bar it implies, a fix-later path test, and the staging and supply implications.
- **Assumptions:** reversibility can be judged from the cost and time of undoing.
- **Tradeoffs:** extra testing and gates before an irreversible step cost time and prevent costly mistakes.
- **Failure modes:** treating hardware like software; a fix-later path that depends on the broken part; no gate on the lock.
- **Worked example:** Worked example 3.

Class	Example	What to do
Reversible	A flagged feature	Move fast; monitor; roll back
Hard to reverse	A public price, an announcement	More evidence; stage; prepare the retraction
Irreversible	A manufactured part, a safety-critical release	Highest bar; gate the lock; validate the fix-later path; consider safety first

FW-C17-04 · The launch-or-hold decision table · Original

- **Inputs:** an experiment readout, the pre-set rule, the cost of delay and the reversibility.
- **Outputs:** one of five actions (launch, staged launch with holdout, extend, fix and retest, do not launch) with the reason.
- **Assumptions:** the rule was fixed before the data (FW-C14-05).
- **Tradeoffs:** partial options reduce regret and add complexity.
- **Failure modes:** a binary choice; ignoring the cost of delay; overriding a guardrail silently.
- **Worked example:** Worked example 2 and C17.4.

Readout	Action
Primary clears the threshold; guardrails within margins; no validity issue	Launch (stage if risk warrants), keep a holdout
Primary positive but interval straddles the threshold; guardrails fine	Staged launch with holdout, or extend
A guardrail interval breaches its margin	Investigate; fix and retest; do not launch as is
Primary below threshold	Do not launch
Validity failure	Rerun

FW-C17-05 · The rollback trigger sheet · Original

- **Inputs:** the metrics that signal harm and their noise levels.
- **Outputs:** for each trigger: metric, threshold, window, owner, action; a fast coarse trigger and a slow sensitive one.
- **Assumptions:** the baseline and noise are known (C13, C12).
- **Tradeoffs:** tight triggers cause false rollbacks; loose ones cause late ones.
- **Failure modes:** thresholds inside the noise; no owner; deciding triggers during the incident.
- **Worked example:** Worked example 4.

Metric	Threshold	Window	Owner	Action
--------	-----------	--------	-------	--------

FW-C17-06 · The incident playbook for PMs · Original

- **Inputs:** an alert or a report of harm after launch.
- **Outputs:** the first 15 minutes: confirm impact, contain, communicate, decide, log; then a blameless postmortem with actions.
- **Assumptions:** an incident lead exists; the PM's role is impact, decisions and communication.
- **Tradeoffs:** a fast rollback may hide the cause; keep logs and a reproduction.
- **Failure modes:** debating cause before containing; silence from the PM; blame in the postmortem.
- **Worked example:** IQ-C17-07.

Steps: **Confirm impact** → **Contain** → **Communicate (cadence)** → **Decide tradeoffs** → **Log** → **Postmortem (blameless, with owners)**.

FW-C17-07 · The dependency and delivery-risk map · Original

- **Inputs:** the launch plan and its dependencies.
- **Outputs:** a table of dependencies with probability on time, impact, mitigation and start date; the compounded probability; the order of scope cuts.

- **Assumptions:** dependencies are independent enough for a first estimate.
- **Tradeoffs:** a map costs an hour a week and avoids late surprises.
- **Failure modes:** treating each dependency as fine; compounding ignored; raising risks late.
- **Worked example:** Worked example 5.

Dependency	Needed by	P(on time)	Impact if late	Mitigation	Owner
------------	-----------	------------	----------------	------------	-------

Compounded probability: **multiply the probabilities of the independent dependencies on the critical path.**

FW-C17-08 · The post-launch review · Original

- **Inputs:** the launch, its success criteria and baselines.
- **Outputs:** reviews at day 1, week 1 and week 4: health, metrics against targets, guardrails, novelty check, holdout; a decision to iterate, scale or stop; a written launch review.
- **Assumptions:** success criteria were written before launch.
- **Tradeoffs:** time spent reviewing against shipping the next thing.
- **Failure modes:** moving the goalposts; reading week one as steady state; no written learning.
- **Worked example:** the review plan in IQ-C17-04 and EX-C17-09.

4. Answer architecture

The launch answer, for a 25-minute round

Step	Minutes	What to produce
Clarify the launch	3	What, who, where, stakes, reversibility; what is fixed
Readiness criteria	4	The gate by category, owners, the go or no-go decider
Rollout plan	5	Ladder with stage sizes, dwell times and advance conditions; flags and canary
Triggers and decisions	5	Rollback triggers with thresholds and windows; the launch-or-hold rule for mixed results
Risks and dependencies	4	The dependency map and compounding; hardware or irreversible elements; scope-cut order
Measurement and iteration	3	Day 1, week 1, week 4 review; iterate, scale or stop
Slack	1	

Pacing rules. If the interviewer gives a specific scenario (a regression, a defect), spend the time on that and use the structure briefly. If you run behind, keep the gate, the ladder and the triggers.

How to say it. State the criteria early: "I'd launch when [criteria] are met, decided by [owner], and I'd expose it in stages: 1%, 10%, 50%, 100%, with a rollback if [trigger]."

How to close. Two sentences: "I'd define readiness with owners and a named decider, ramp in stages sized to catch the harm I fear, set rollback triggers before launch, and review at day one, week one and week four. I'd change the plan if [signal]."

Type: walk me through a launch

Use the full structure.

Type: the metric regressed after launch

Use the incident playbook (FW-C17-06) and the diagnosis spine (C13): confirm, contain by trigger, locate, decide, communicate.

Type: launch or hold given results

Use the decision table (FW-C17-04) and the pre-set rule; quantify the cost of delay.

Type: hardware or irreversible launch

Use the irreversibility check (FW-C17-03): lock date, fix-later path, supply, certification, safety.

Type: engineering says three times longer, or a slipping date

Use the dependency map and scope cuts (C17.6, C16.4, C15.4).

Type: define success for X

Use the success-metric template (C05) and the post-launch review.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"When is it ready?"	Readiness	Give criteria, owners and the decider, not a date
"Why 1%?"	Stage sizing	Show what harm that size can detect
"What makes you roll back?"	Triggers	Give metric, threshold, window and owner, with noise
"The test is mixed."	Launch or hold	Apply the pre-set rule; offer a staged option
"Can't you fix it in software later?"	Hardware	Test the update path; reversibility; the lock date
"It's 2 a.m. and errors are up."	Incident	Confirm, contain, communicate, then diagnose

Interviewer says	What it means	What to do
"How will you know it worked?"	Measurement	Day 1, week 1, week 4; criteria set beforehand

5. Worked examples

All five examples are **Illustrative**: the products are fictional or use a product type as the setting only, and every figure is an invented assumption. The arithmetic was computed. Each is written as notes you could say aloud.

Worked example 1 — Sizing a rollout ladder to catch the harm you fear (Illustrative)

The situation. A web product with 1,000,000 users (invented) is releasing a rewritten account page. The team's main worry is a rise in errors. The error rate is 1% today.

What each stage can see (FW-C17-02). Comparing the exposed users with everyone else, 80% power at the 5% level:

Rise in error rate to detect	Users needed in the stage	Share of 1,000,000
+0.5 points (1.0% to 1.5%)	about 4,700	0.5%
+0.3 points	about 11,300	1.1%
+0.1 points	about 93,400	9.3%

A 1% canary (10,000 users) can reliably see a rise of about 0.33 points or more. To see 0.1 points the stage must be about 9% of users. At a 1% error rate, 0.1 points is 1,000 extra failures per million users.

The ladder.

Stage	Exposure	Dwell	What it can detect	Advance if
Internal	Staff	1 day	Gross failures, wrong data	No S1; events logged correctly
Canary	1%	1 business day	Severe harm (about +0.33 points of errors or more)	No trigger hit; errors within 0.3 points of baseline
Early	10%	2 days	Subtle harm (+0.1 points) and slow problems (memory growth over hours)	Errors within 0.1 points; latency within margin; support contacts normal
Broad	50%	3 days, covering a weekend	Weekly patterns; rare paths	Same
Full	100%			Keep a 2% holdback for two weeks to compare

Reasoning behind the stages. The canary protects against a severe failure at a small blast radius: if the page breaks outright, 10,000 users are affected for a day at most, with a flag to turn it off. The move to 10% is what protects against subtle harm, because only at that size can a 0.1-point rise be seen. Dwell times cover a business day (the daily cycle) and then a weekend; some defects appear only at scale or over hours.

What the ladder cannot see. A change in conversion of a fraction of a percent needs far more users than a rollout reaches in a few days, so a rollout watches for harm, and the question of benefit is for an experiment with a holdout (C14).

What this shows. Stage sizes follow from the smallest harm you will not accept, and the arithmetic shows why a 1% canary is not the same as safety against every problem.

Worked example 2 — Launch or hold with mixed results (Illustrative)

Setting: a streaming service of the kind Netflix operates. No fact about any real product is claimed.

The readout. A test of a new recommendation approach ran one week on 10,000,000 members (invented). The rule, fixed beforehand (FW-C14-05): launch if the primary interval's lower end is at or above the threshold and all guardrails are within margins.

Metric	Result	95% interval	Rule
Viewing hours per member (primary)	+1.5%	+0.3% to +2.7%	Threshold +1.0%: lower end below it
30-day retention (guardrail)	−0.2 points	−0.8 to +0.4	Margin −0.5: lower end breaches
Support contacts (guardrail)	0%	−2% to +2%	Margin +3%: within

Apply the order (FW-C14-07, FW-C17-04). Validity passes. Guardrails: retention's lower bound (−0.8) is outside the margin, but the interval includes zero and positive values; it is uncertainty, not evidence of harm. Primary: the estimate (+1.5%) is above the threshold, and the interval straddles it. By the strict rule: do not launch as is; extend.

What would extending take? The retention interval has a width of 1.2 points. Halving it to ± 0.3 needs four times the data. If a week gave this, four weeks in total, so three more. During those three weeks, the test could run at a larger exposure: that is itself a staged launch, with a holdout, and the guardrail watched daily against the margin.

Cost of delay. If the true gain is +1.5% in viewing, and it is worth, say, \$4,000,000 a year across the member base (invented), three weeks of delay costs about \$230,000. Against the risk of a lasting retention loss: a 0.5-point fall in 30-day retention across 10,000,000 members is 50,000 members, at a value of, say, \$100 each, \$5,000,000. The cost of waiting is small beside the downside, which is why the extension is justified.

Novelty check. A week is also too short to rule out novelty (C14.4). The extension helps with that too: the effect by week should settle.

Decision. Stage it: expand to 25% of members with a 5% holdout for three more weeks; launch fully if the primary lower bound clears +1.0% and retention's lower bound is above -0.5; otherwise do not launch or redesign. Tell the team the rule and the dates now.

What this shows. The decision followed a rule written first, the arithmetic of delay against downside, and a partial option that turned "wait" into "learn while limiting exposure."

Worked example 3 — A defect two weeks before manufacturing locks (Illustrative)

Setting: a wireless accessory of the kind Apple and several other companies make. No fact about any real product is claimed.

The situation. An accessory will be manufactured in a first run of 500,000 units. Manufacturing locks in 14 days: after that, changes to the device require a delay of weeks and cost millions. Testing finds a defect in the device's firmware: in 4% of units in lab tests (invented), the accessory loses its connection to the phone intermittently. Firmware can be updated over the air if the device can connect. A delay of the lock by 14 days would, the team estimates, miss the holiday sales window and cost \$3,000,000 of margin (invented).

Classify (FW-C17-03). The manufactured part is irreversible; the firmware is changeable only through the update path. So the question is whether the fix-later path is real.

The circular risk. The update needs a working connection. The defect is a connection loss. If the affected units cannot reliably receive the update, "fix it later in software" fails exactly when it is needed. So the first test is not "can we write a fix" but "can an affected unit receive it."

Options and numbers. 4% of 500,000 is 20,000 units. A field problem costs about \$60 per affected unit (returns, support, replacement; invented).

- *A: Lock on time with a gate.* By day 7, test the update path on 200 affected units. Suppose a 0.8 chance it works on at least 90% of affected units. If it works, residual defects are 10% of 20,000, 2,000 units, costing \$120,000. If it fails (0.2), delay the lock by the remaining 7 days at a cost of \$1,500,000 (half the full delay, since the schedule has absorbed a week). Expected cost: $0.8 \times \$120,000 + 0.2 \times \$1,500,000 = \$96,000 + \$300,000 = \$396,000$.
- *B: Delay the lock 14 days now.* Cost \$3,000,000 for sure.
- *C: Lock on time and ship as is.* $20,000 \times \$60 = \$1,200,000$, plus a reputational cost.

On expected cost A (\$396,000) beats C (\$1,200,000) and B (\$3,000,000). The value of A is the gate: it keeps the option to delay while gathering the one piece of evidence that decides.

Supply and the rest. Check also whether the first run is sized for a launch with possible returns, whether units from the first weeks can be segregated and updated before shipping, and whether the update can be pushed from day one so most customers get the fix on first connection. Brief support on the symptom and the workaround. Certification: confirm that a

firmware change does not require repeating approvals [V: FCC equipment authorization, 47 CFR Part 2 and section 2.1043 on firmware (permissive changes); US rules for radio devices only; safety approvals and non-US regimes not checked].

Decision. Choose A. Run the update-path test by day 7 on the affected units, with a stated criterion (updates succeed on at least 90% of affected units), and decide on day 7 whether to lock or to delay. If the defect were a safety risk, the options table would change: safety is not traded against cost.

What this shows. For an irreversible step, the work is to create an early test that decides, so that the choice to lock or delay is made with evidence while it is still possible to delay.

Worked example 4 — Rollback triggers that respect noise (Illustrative)

The situation. A checkout redesign launches to 10% of users. Payment success is the key metric, with a baseline of 96% (invented) and about 1,000 attempts per half hour in the exposed group.

Noise first (FW-C17-05). Over a half hour, the standard error of the success rate is $\sqrt{(0.96 \times 0.04 \div 1,000)} = 0.62$ points. A drop of 1.0 point is only 1.6 standard errors: the chance of seeing a drop that large by luck in a given window is about 5%. With 48 half-hour windows a day, that is about 2.6 false alarms a day. A trigger of "success below 95% in any half hour" would fire by chance almost every few hours and teach people to ignore it.

Over two hours, with 4,000 attempts, the standard error is 0.31 points, and a 1.0-point drop is 3.2 standard errors: a chance of about 0.06% per window, with 12 windows a day, under 1% of days with a false alarm. So the slow trigger is reliable and the fast one is not.

The trigger sheet.

Metric	Threshold	Window	Owner	Action
Payment success (exposed users)	Below 95.0%	Rolling 2 hours	On-call engineer	Turn off the flag; page the PM
Payment success	Below 92.0%	Any 15 minutes	On-call engineer	Turn off the flag immediately
Checkout error rate	Above 2.5%	Rolling 1 hour	On-call engineer	Turn off the flag
Support contacts about payment	Above 1.5× baseline	Rolling 4 hours	Support lead, then PM	Pause the ramp; review
Any payment of the wrong amount	One confirmed case	Immediate	PM and engineering lead	Turn off the flag; incident

Why two thresholds for one metric. A large, fast drop (below 92% in 15 minutes) is clear even in a small window because the effect is far larger than the noise; a smaller, slower drop needs the longer window. Together they cover both severe and subtle failures. The last trigger is a single event, not a rate: some failures (money wrong) are unacceptable in any number.

What the PM does. Agree the sheet with engineering and support before the launch, put it where the on-call can see it, and agree that anyone on-call can act on a hard trigger without asking.

What this shows. Triggers set without noise either cry wolf or sleep through the fire, and settled in advance they remove a debate from the worst moment.

Worked example 5 — Delivery risk compounds (Illustrative)

The situation. A launch depends on five things from outside the core team (invented): legal sign-off, a partner's API, design assets, a security review and localization.

The map (FW-C17-07).

Dependency	P(on time)	Impact if late	Mitigation
Legal sign-off	90%	Blocks launch	Submit the brief now; weekly check
Partner API	85%	Blocks the main feature	Build against a mock; agree a fallback
Design assets	95%	Delays polish	Ship with placeholders
Security review	90%	Blocks launch	Book the slot now
Localization	92%	Delays some markets	Launch in the first market

The compounding. The chance that all five arrive on time is $0.90 \times 0.85 \times 0.95 \times 0.90 \times 0.92 = 0.60$. A plan that reads as five "probably fine" items has a 40% chance of slipping.

What to do. Raise the weak ones: building against a mock and agreeing a fallback lifts the partner API item to 93%; booking the security review now lifts it to 95%. The new product: $0.90 \times 0.93 \times 0.95 \times 0.95 \times 0.92 = 0.70$. Reduce the count on the critical path: launch in the first market so localization does not block. Add buffer where risks cluster (legal and security together: allow a week). Start the longest-lead items first.

Scope cuts, in advance. If the date is threatened, cut in this order: placeholders instead of final assets; fewer markets at launch; a smaller first feature set. Never cut the launch criteria (quality bar, accessibility, security).

Escalate early. Review the map weekly. A dependency that is slipping is raised that week with options, not on the day it fails.

What this shows. The probability arithmetic converted vague worry into a 60% number, and each mitigation had a measurable effect on it.

6. Baseline, Strong, Standout

Prompt: "You are launching a redesigned checkout in six weeks. Walk me through it."

Baseline answer

"I'd make sure engineering finishes on time, do testing, and then release it to everyone with a marketing push. If something goes wrong, we'd fix it."

What is wrong: readiness is the date, the release is all at once, and "fix it" is the rollback plan. There are no criteria, no stages, no triggers, no measurement.

Strong answer

"I'd define readiness with criteria and owners: acceptance criteria met, no open severe defects, accessibility checked, instrumentation working, support trained, legal and security sign-offs. One person decides go or no-go.

"I'd roll out in stages behind a feature flag: staff first, then 1%, 10%, 50%, 100%, with a minimum time at each stage and conditions to advance. I'd set rollback triggers before launch, for example payment success below a threshold over a rolling window, and say who acts.

"I'd measure the primary metric, completed purchases, with guardrails on errors, speed and support contacts, and review at day one, week one and week four. I'd keep a small holdout to see the true effect."

Standout answer

Everything in the Strong answer, plus:

"I'd size the stages to the harm I fear: a 1% canary on a million users can see a rise of about a third of a point in errors, but to see a tenth of a point I need about 9% exposure, so the move to 10% is the real safety stage. I'd set the triggers with the noise in mind: a half-hour window on a 96% payment success rate has a standard error of about 0.6 points, so a one-point drop could be luck, and I'd use a two-hour window for the sensitive trigger and a 15-minute window for a large drop.

"I'd map the dependencies and compound them: five dependencies at 85 to 95% give about a 60% chance all arrive on time, so I'd raise the weakest, reduce the number, and decide the order of scope cuts in advance. If the launch date is fixed, I'd cut scope or exposure before I cut verification.

"For mixed experiment results I'd use the pre-set rule, and prefer a staged launch with a holdout to a binary choice. For the first week I'd expect novelty, so I'd read the steady state in weeks three and four.

"What would change my plan: any safety or money-handling defect, which pauses the ramp regardless of metrics, or evidence that the canary is not representative (for example, only new users)."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
Criteria, owners and a named decider; a staged ramp with a flag	Stage sizes chosen to detect the harm; triggers set with noise in mind
Rollback triggers and a post-launch review	Compounded dependency risk; scope-cut order; a rule for mixed results
Guardrails and a holdout	Names what would pause the launch and what the canary cannot see

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10].

Company lens: Apple [as of 2026-10]. Apple ships hardware, software and services together, and hardware cannot be recalled by an update Public [V: Apple Form 10-K for fiscal 2025 (products and services); the recall point is the author's reasoning]. Third-party guides report that launch prompts at Apple include scenarios in which a defect is found close to a manufacturing deadline Inferred. Practical consequence: classify the decision's reversibility, test the fix-later path, quantify delay against field cost, and treat safety defects as outside the tradeoff.

Company lens: Amazon [as of 2026-10]. Amazon's leadership principles include Bias for Action, and its founder has described decisions as one-way or two-way doors, with speed on the two-way ones Public [V: Amazon's Leadership Principles; Bezos 2015 shareholder letter; the latency claim was not rechecked]. A widely cited claim that small increases in page latency reduced sales at Amazon dates from the mid-2000s and its current size is not established Public [V: Greg Linden, Make Data Useful, Stanford talk slides, 29 November 2006: 100 ms extra load time, 1 percent drop in Amazon sales; cited by Kohavi and Longbotham, 2007; the size today is not established]. Candidates commonly report launch questions weighing a conversion gain against latency or reliability Inferred. Practical consequence: measure the cost of latency in your own data, and move quickly on reversible launches while gating irreversible ones.

Company lens: Google [as of 2026-10]. Google's published work on site reliability describes canarying, staged rollouts and blameless postmortems Public [V: Google SRE book, 2016, and sre.google]. Candidates commonly report scenarios about launching at scale Inferred. Practical consequence: describe a canary, a ramp and an automatic rollback, and say how you would read a regression at large scale.

Company lens: Meta [as of 2026-10]. Meta has published an account of a large 2021 outage and its recovery, a case of a configuration change and the difficulty of rolling back Public [V: Meta engineering post of 5 October 2021]. Candidates commonly report scenarios in which a launch hurts a metric and the team must decide whether to roll back Inferred. Practical consequence: have rollback triggers and the ability to act when your usual tools are affected.

Company lens: Microsoft [as of 2026-10]. Microsoft publishes post-incident reviews for cloud outages and has described safe deployment practices Public [V: Azure status history publishes post incident reviews; safe deployment practices not rechecked]. Practical consequence: for enterprise products, stage by customer group, include design partners, and communicate to administrators before changes.

Company lens: Netflix [as of 2026-10]. Netflix has published on automated canary analysis and on resilience testing Public [V: Netflix Tech Blog posts on experimentation and resilience testing]. Practical consequence: describe automated comparison of the canary with a control, and say what metrics decide.

Company lens: Stripe [as of 2026-10]. Stripe publishes a status page and documents API versioning so that changes do not break integrations Public [V: Stripe documentation; the status page was not opened]. Practical consequence: for products others build on, version changes, give notice, and treat backward compatibility as a launch criterion.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	The date is the goal	"We launch Friday" with no criteria	Write readiness criteria and a go or no-go owner
2	Releasing to everyone at once	No flag or staging	Use a flag and a ladder
3	A canary too small to see the harm	1% for a subtle metric	Size stages to the smallest harm you reject
4	Rollback decided mid-incident	Debate at 2 a.m.	Agree triggers, owners and actions beforehand
5	Triggers inside the noise	Alerts fire constantly	Use the noise band and longer windows
6	Treating hardware like software	"We'll patch it"	Test the update path; gate the lock
7	A binary launch decision	All or nothing on mixed results	Use staged launch, holdout or extension

#	Mistake	How to detect it	Fix
8	Ignoring compounding dependencies	Each "probably fine"	Multiply; reduce, raise and buffer
9	Waiving criteria silently	"Just this once"	Only the decider, in writing
10	No post-launch review	The team moves on	Day 1, week 1, week 4, against pre-set criteria

9. Mental models applied

Four models, each tied to a decision in this chapter. [[pre-mortem]] and [[reversible-decision]] are from C02.

The pre-mortem (MM-pre-mortem, see [[pre-mortem]], C02). Before launch, assume it failed and ask why. The answers become the checklist and the triggers. A team that imagines the three most likely failures (a payment error, a slow page, a confusing label) has already decided what to watch.

Swiss cheese (MM-swiss-cheese). Accidents happen when holes in several layers of defense line up [V: Reason, Managing the Risks of Organizational Accidents, 1997]. Testing, staging, monitoring, triggers and rollback are layers, each with holes. Do not rely on one layer; add layers that fail differently, and be most careful where several holes could line up (a rushed schedule, a new dependency and a skipped review).

One-way and two-way doors (MM-reversibility, see [[reversible-decision]], C02). Move fast through doors you can walk back through; slow down at the ones you cannot. Flags make most software launches two-way; manufactured parts, public commitments and data deletion are one-way.

Normalization of deviance (MM-normalization-of-deviance). When small departures from a standard go unpunished, they become the norm until a failure exposes them [V: Vaughan, The Challenger Launch Decision, 1996]. A launch criterion waived "just this once" makes the next waiver easier. Waive in writing, rarely, and review the waivers.

Engineering, design and data-science perspectives

- **Engineering and reliability.** Engineers own the release mechanism, monitoring and rollback. Ask what the off-switch is, how long it takes, what it cannot undo (data), and who is on call. Rehearse the rollback before launch. Agree who may act on a hard trigger without asking.
- **Design and research.** Watch real users in the first days: a flow that tests well can fail in the wild. Plan a way to collect feedback from the canary and to fix usability problems before the ramp widens.
- **Data science.** Agree the success criteria, the holdout, the instrumentation checks and the trigger thresholds with the noise in mind. Ask for dashboards that show the exposed and unexposed groups side by side.

- **Support, legal, security and communications.** They need to know what is changing and when, and what to say. Give them the stage schedule, not just the launch date, so they can staff and respond to each step.

Checklist: before you ship

1. Are the readiness criteria written, with owners and a named go or no-go decider?
2. Is there a flag or equivalent, and has the rollback been rehearsed?
3. Are the stages sized to the smallest harm I would reject, with dwell times and advance conditions?
4. Are the rollback triggers set, with thresholds and windows that respect noise, and owners who may act?
5. Is the rule for mixed results fixed, and is there a holdout?
6. Have I classified reversibility and, for anything irreversible, validated the fix-later path?
7. Have I compounded the dependency risk and set the order of scope cuts?
8. Are the day-one, week-one and week-four reviews booked, against criteria written beforehand?

10. Case studies

CS-C17-01 · The Friday launch with no way back · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. At 16:00 on a Friday, a team releases a redesigned search page to 100% of users. The team has tested it for two weeks and is confident. There is no feature flag; the only way back is to redeploy the previous version, which takes about 90 minutes once someone is found to do it.

Constraints. A date promised to a partner, no staging in the release process, and a weekend with a small on-call rota.

Data. The product has about 50,000 search sessions an hour on a Friday evening. Within 20 minutes of launch the error rate on the new page is 6% (against under 1% before). Support contacts rise, but there is no alert on the error rate. The on-call engineer notices at 19:00 when support escalates; finding the people who can approve and perform a redeploy takes until 20:30; the old version is back at 22:00. The page was failing for 6 hours in total.

Problem definition. As an observable problem: "A change that failed for 6% of sessions was exposed to every user for six hours because there was no staging and no pre-agreed way back."

Analysis. The failure itself (a bug found only at scale) is ordinary. What turned it into an incident was the exposure and the recovery. At 50,000 sessions an hour and a 6% error rate, 3,000 sessions failed each hour, 18,000 over six hours. Compare a 1% canary with a flag: 500

sessions an hour, 30 failures an hour at 6%, and a trigger that fires in 30 minutes would have exposed about 15 failed sessions before the flag went off: roughly 1,200 times fewer. The launch gate (FW-C17-01) would have asked for a rehearsed rollback, a named decider on the weekend and rollback triggers; none existed. The 90-minute redeploy was itself a risk the checklist should have caught.

Options.

Option	Optimizes	Costs
A. Keep releasing to everyone on Fridays	Speed	Large blast radius; weekend incidents
B. Add a flag, a 1% to 10% to 100% ladder and rollback triggers; avoid Friday afternoons for risky changes	Small blast radius; fast recovery	A few days of tooling; slower to full release
C. Add more pre-release testing only	Fewer bugs	Cannot find everything that only appears at scale

Tradeoffs. B costs a little speed and tooling and cuts the worst case sharply. C helps and does not replace B. A repeats the incident.

Decision. Choose B, with C as a complement. The release process gains a flag and a ladder (1% for a business day, then 10%, 50%, 100%), triggers on error rate and checkout completion with named owners, a rehearsed rollback that takes under five minutes, and a rule that risky releases do not go out on Friday afternoons. Primary measure: time to detect and time to recover in the next incident, plus the number of users exposed to a defect before rollback.

Outcome. In this fictional scenario, three months later a defect in a different feature was caught at the 1% stage within 25 minutes, the flag was turned off by the on-call engineer, and about 20 users saw errors (invented). The postmortem credited the ladder and the trigger sheet.

Alternative outcomes. If the canary had been too small to see a subtle bug, the 10% stage would have caught it, as Worked example 1 shows. If the trigger had been set inside the noise, it would have fired falsely and been ignored. The lesson: staging, triggers and a rehearsed way back change the cost of an ordinary bug from a weekend incident to a small blip.

CS-C17-02 · Samsung Galaxy Note 7, 2016: a hardware defect, a recall and a second failure · Verified · Advanced

Sources (types): Samsung's own Newsroom page and product pages on its press conference of 23 January 2017 (primary); Associated Press reports of that announcement and of Samsung's cost statement of 14 October 2016 (carried by CBC and other outlets); Reuters' reports of the September 2016 recalls (carried by Fortune); Forbes' account of the investigation; and trade-press reports of the December 2016 software update. Samsung's own announcements are the primary sources for its decisions and findings. The analysis is the author's. The sources do not reveal Samsung's internal deliberations. Re-read in October 2026.

Situation. Samsung released the Galaxy Note 7 in August 2016. After reports of batteries overheating and catching fire, on 2 September 2016 it suspended sales and announced a replacement program for about 2.5 million phones, followed by a formal recall of about 1 million phones in the United States on 15 September [V: Reuters via Fortune, September 2016]. Replacement phones then also caught fire in some cases, including one aboard a US passenger flight in October. On 10 October Samsung stopped sales and exchanges worldwide, and on 11 October it ended production permanently. In October 2016 Samsung said that discontinuing the phone would cost about \$3 billion over the October to March quarters, bringing the total cost of the recalls to at least \$5.3 billion [V: Samsung statement of 14 October 2016, as reported by AP]. In January 2017 it announced the results of an investigation: design and manufacturing errors in two different battery types made by two different manufacturers. It said it took responsibility for its failure to ultimately identify and verify the issues in the battery design and manufacturing process.

Constraints. A flagship device in tens of millions of potential hands is an irreversible release: the defect is in the hardware, in a sealed battery cell, which software cannot fix; a physical recall requires finding and exchanging millions of units. The product had been launched with high expectations and a competitive timing against a rival's launch (reported). Visual inspection could not reveal the defects, which were inside the cells (reported).

Data. As reported: the dates above; roughly 2.5 million units in the first recall; about 3 million Note 7 devices sold, of which more than 96% had been returned as of 23 January 2017 [V: Samsung page]; an investigation by 700 researchers and engineers on more than 200,000 devices and more than 30,000 batteries [V: Samsung and AP]; independent firms (UL, Exponent, TÜV Rheinland) examining batteries and the supply chain [V: Samsung and AP]. From 19 December 2016 Samsung released a software update that prevented remaining US devices from charging, to increase returns, which T-Mobile, AT&T and Sprint scheduled between 27 December and 8 January and which Verizon first declined and then agreed to push [V: trade press, December 2016]. Samsung was criticized for its handling of the recall and for a hasty initial investigation, according to AP.

Problem definition. *The author's reading.* A safety-relevant defect reached customers because verification before launch did not catch it, and the response to the first discovery did not establish that the fix was safe. As an observable problem: "Phones that could catch fire were sold, and the replacements, which were meant to be safe, also failed in some cases."

Analysis. *The author's analysis.* Read through this chapter, the case is a study of irreversibility. *Irreversibility check (FW-C17-03):* a manufactured, sealed battery in a consumer device is at the far end of the scale; the bar for evidence before the lock and before shipping should be the highest, and safety is not a tradeoff against revenue. *Fix-later path:* the software update could not fix a battery cell. It could only disable charging, a containment measure, not a remedy, and it was used to increase returns. *Verification of the fix:* the first remedy, replacement units with batteries from a different supplier, were themselves reported to have failed in some cases, and the investigation later found that the second supplier's batteries also had a defect. The lesson is not that replacement is wrong, but that a replacement is a new release and needs its own verified readiness, with a rollback-equivalent (stop sales) triggered by the first signal. *Triggers and incident response:* the

October reports of replacements failing led within days to a halt of sales (10 October) and production (11 October); the pre-agreed trigger for halting, if there was one, is not in the sources. *Blast radius*: with millions of units sold, the staged-release instruments of software (flags, canaries) were not available; the nearest analogue is staged availability by region and a field trial before mass production, which may have shrunk the number of units in customers' hands at the time of discovery.

The case is not evidence about any one decision-maker's competence. It shows the structure of the risk: a hardware defect plus compressed verification plus a large volume made the cost of a mistake very large.

Options. *The author's reconstruction of choices a team could have faced; not a record of Samsung's deliberations.*

Option	Optimizes	Costs
A. Recall and replace quickly with a modified battery, as announced on 2 September	Speed; limiting units at risk	Risk that the replacement is unverified
B. Stop sales and exchanges until the root cause is identified and the replacement is verified safe	Safety and certainty	A longer gap; a stronger competitor position; cost
C. End the product early once the cause is unclear	Stops further exposure	Loses the flagship line

Tradeoffs. A limits exposure quickly and relies on the cause being understood. B delays the remedy until it can be trusted. C is decisive and expensive. When safety is at stake, an unverified remedy is a second launch, with the same irreversibility.

Decision. As documented, Samsung chose A on 2 September, and, after replacement incidents, moved to halt sales and exchanges on 10 October and end production on 11 October.

Outcome. Reported: about 3 million devices sold and more than 96% returned by 23 January 2017, an estimated cost of at least \$5.3 billion (October 2016), the discontinuation of the model, the January 2017 findings, and a statement taking responsibility [V: Samsung; AP]. Later effects on the brand and the company's practices are outside the sources used here.

Alternative outcomes. *Inference.* Had the root cause been established before the replacement program began, the second failure might have been avoided; had a smaller early field trial been used, fewer units would have been exposed. A reader should keep in mind the limits of hindsight: the investigation took months and the pressure to respond was immediate. The method to carry away: treat an irreversible release as the highest-stakes launch, verify any remedy as a new launch, set the stop trigger before you need it, and put safety outside the tradeoff.

11. Interview questions and model answers

Seven questions: two Beginner, two Intermediate, three Hard. Model answers are **Illustrative** (composite candidates), every figure is an assumption, and the arithmetic was computed. Each is written against the six behaviors in section 1. Where a real company's product is the setting, the answer reasons from public information and states no internal fact. The answers are fuller than a first pass in a real round.

IQ-C17-01 · Beginner · execution · EX

Prompt. What does "ready to launch" mean to you?

Rubric.

- **Baseline:** "The code is done and tested."
- **Strong:** Lists criteria across product, reliability, data, support and constraints, with a decider.
- **Standout:** Scales the gate to the stakes, requires a rehearsed rollback and written success criteria, notes that only the decider may waive a criterion and in writing, and links readiness to a staged plan.

Model answer. Ready means a short list of criteria is met, each with an owner, and one named person has decided go. A date is not readiness.

The criteria. Product: acceptance criteria met, defects triaged by severity, no open S1, accessibility checked. Reliability: load tested, monitoring and alerts live, and a rollback or off-switch rehearsed, because a rollback that has never been tried is a hope. Data: events logged and verified in production, a dashboard and a baseline, and success criteria written before launch. People: support trained with scripts, sales and success informed, internal and external messages ready. Constraints: legal, privacy and security sign-offs recorded and any open risk accepted in writing by a named owner. Rollout: stages, dwell times and rollback triggers agreed.

The decider. One person, usually the PM with the engineering lead, makes the go or no-go call at a meeting a few days before the date, with the checklist in front of them and dissent recorded.

Scale it. A copy change behind a flag needs a light gate; a payments change needs the full one. And I'd write the criteria in week one so the date does not bend them in week six. A criterion may be waived only by the decider, in writing, so the exception is visible and rare.

The mistake to avoid is "launch Friday" as the goal. If testing finds problems on Thursday, the right move is a smaller launch or a later date, chosen by the criteria and not by the calendar.

Follow-ups.

- **Q.** Who can waive a launch criterion, and how? **A.** "Only the person who owns the go decision, in writing, with the reason, the risk accepted, an owner for follow-up and a date by which the criterion will be met. A PM or an engineering lead cannot waive a safety or

data-loss criterion alone, and a waiver given in a hallway is not a waiver. Recording it protects the team: when something goes wrong, the question is whether the risk was seen and accepted by the right person, and the waiver log shows which criteria are habitually waived, which tells us the criteria need changing."

- **Q.** How does readiness link to a staged plan? **A.** "Each stage has its own gate. For example: staff at launch minus two weeks; 1% of users for two days; 10% for three days; 50% for three days; then everyone. A stage advances only if guardrails hold: error rate, latency, support contacts and the primary metric within bounds, with the rollback rehearsed before the 1% step. Readiness is then a series of small decisions with evidence, not one big bet on a date, and it lets me tell stakeholders exactly what we know at each step."

IQ-C17-02 · Beginner · execution · EX

Prompt. What is a feature flag, and why should a PM care?

Rubric.

- **Baseline:** "It turns features on and off."
- **Strong:** Explains staged exposure, fast rollback and experiments.
- **Standout:** Covers canary and ramp use, kill switches, testing in production with a small audience, the cost (complexity, flag debt) and what a PM does with them.

Model answer. A feature flag is a switch that turns functionality on or off for chosen users without shipping a new release. The code for the feature is deployed, but who sees it is controlled by the flag.

Why a PM cares. First, *staged rollout*: I can show a change to staff, then 1% of users, then 10%, and widen only if it behaves, which limits the blast radius. Second, *fast rollback*: if something goes wrong, switching the flag off takes seconds, compared with an hour or more to redeploy. Third, *experiments*: flags assign users to variants, so I can compare an exposed group with a control (C14). Fourth, *decoupling release from launch*: engineering can ship code when it is ready and the business can choose the moment to turn it on, with marketing and support aligned. Fifth, a *kill switch* for risky components.

Costs. Flags add complexity: more code paths to test, and flags that stay after a launch become debt, so each flag needs an owner and a removal date. A flag is not a replacement for testing: it limits the damage of a mistake, it does not prevent it.

What I do with them. Agree with engineering which launches use flags, who can flip them, the stages and the rollback triggers, and make sure the off-switch is tested before launch. Where a change cannot be flagged (a data migration, a hardware design), I treat it as higher risk and gate it harder.

Follow-ups.

- **Q.** What are the costs of feature flags? **A.** "Complexity and debt. Every flag multiplies the combinations of behavior that exist in production, some of which were never tested together, and stale flags left in the code make it harder to understand and change. Flags

can also be left on for a segment indefinitely so that two products coexist. Mitigations: each flag has an owner and an expiry date, a flag is removed within a sprint or two of full rollout, and a list of live flags is reviewed monthly. The benefit of control is worth it only if the flags are cleaned up."

- **Q.** What does a PM do with flags? **A.** "Decide the ramp plan and the guardrails: who sees the change, in what order and for how long, which metrics must hold at each step, and who has authority to turn it off. Use flags for comparison: the exposed and unexposed groups give a clean read on the effect. Ask engineering how fast the kill switch works and test it before launch, and set the date for removing the flag. A flag is a way to reduce the cost of being wrong, and it only works if the rollback is rehearsed."

IQ-C17-03 · Intermediate · execution · EX · Amazon

Prompt. A test of a checkout change shows orders up 0.4%, but the page loads 80 milliseconds slower. Do you launch?

Rubric.

- **Baseline:** "Yes, orders are up," or "No, latency matters."
- **Strong:** Weighs the gain against the cost of latency using the data.
- **Standout:** Measures the product's own latency sensitivity, computes the net under stated assumptions, considers fixing latency and staged launch, segments by device, and sets a decision rule.

Model answer. I'd not decide on the sign of each number. I'd ask whether the 0.4% survives the latency, and I'd use our own data for the cost of slowness.

A rough check. A widely cited claim from the mid-2000s is that each additional 100 milliseconds of latency cost a retailer about 1% of sales [V: Greg Linden, Make Data Useful, Stanford talk slides, 29 November 2006: 100 ms extra load time, 1 percent drop in Amazon sales; cited by Kohavi and Longbotham, 2007; the size today is not established]. That figure is old and context-specific (Kohavi's own later slides describe it as pages slowed by 100 to 250 milliseconds costing about 1% of revenue, so scaling it in a straight line is itself an assumption), so I'd treat it only as a prompt for an assumption. If it held here, 80 milliseconds costs about 0.8% of orders. Net: $+0.4\% - 0.8\% = -0.4\%$. On that assumption, launching loses orders. If our sensitivity were only 0.25% per 100 milliseconds, the cost is 0.2% and the net is $+0.2\%$. So the answer depends on a number I can measure.

Measure the sensitivity. Run an experiment that adds artificial delay to a random group (say +100 ms) and measure the change in orders. That gives our own cost per 100 milliseconds, by device (mobile users on slow connections are often more sensitive).

Look for a better option. Why is the page slower? If the cause is fixable (an uncompressed image, a blocking script), fix it and retest: the 0.4% gain without the latency may be the real result. If the slowdown is inherent, ask whether the gain justifies it on the segments where it is positive.

Decision. Do not launch to everyone on this readout. Fix the latency and retest, or launch in stages to the segments (desktop, fast connections) where the net is positive, with the latency guardrail watched and a holdout. The rule: launch if orders are up by more than the latency cost, with the interval for the net above zero. What would change my mind: a measured sensitivity much lower than the claim.

Company note. Amazon is the setting only: **Inferred** from candidate reports that launch prompts weigh conversion gains against latency and reliability **[unverified: candidate-reported]**. No Amazon data or practice is claimed.

Follow-ups.

- **Q.** How would you use your own data to price the latency? **A.** "Run a latency experiment. Add 100 milliseconds of delay to a random 2% of checkout sessions for a week and compare their order rate with the rest. If orders fall by 0.3% per 100 milliseconds, then 80 milliseconds costs about 0.24%, and the net of the checkout change is $0.4\% - 0.24\% = +0.16\%$. That is a much smaller gain than the headline, and it tells me the latency fix is worth about 0.24% of orders, which at the company's volume is a number I can compare with the cost of engineering time. A published rule of thumb from other companies is only a prompt to measure."
- **Q.** What decision rule would you set? **A.** "Ship if the net after latency is positive and the latency guardrail holds, for example the 95th-percentile checkout latency rises by no more than 50 milliseconds, and fix the latency before launch if it is cheap. I would read the effect by device, since slow phones and weak connections are more sensitive to delay; a net gain on desktop and a loss on low-end phones would argue for shipping the change to desktop first. A staged launch with a latency alarm gives a safe way to proceed."

IQ-C17-04 · Intermediate · execution · EX

Prompt. Two days after launch, sign-up conversion has dropped 3%. What do you do?

Rubric.

- **Baseline:** "Roll back," or "investigate."
- **Strong:** Confirms the drop is real, relates it to the launch, and decides on rollback with the cost.
- **Standout:** Uses the pre-set triggers, applies the diagnosis steps, separates launch effect from other causes, decides with numbers, and communicates.

Model answer. I'd treat it as an incident with a diagnosis attached. First, is it real and is it ours? I'd apply step zero from C13: is the data complete, is 3% outside the normal swing (say daily swings are $\pm 1.5\%$, so 3% is notable), and does the timing line up with the launch (the drop starts at the release hour and not before)? If the launch was staged, compare the exposed group with the unexposed: a drop only in the exposed group is strong evidence the launch is the cause; if both dropped, look for an external cause.

Check the trigger sheet. If a rollback trigger was set ("sign-up conversion down more than 3% over a rolling 24 hours"), and it is hit, the action is pre-agreed: pause the ramp or turn off the flag, then diagnose. If no trigger existed, I'd decide with numbers.

The numbers. With 10,000 sign-ups a day, a 3% drop is 300 sign-ups a day. Suppose each is worth \$10 in lifetime margin (invented): \$3,000 a day. The launch must have been expected to add more than that to be worth keeping. If the benefit is not yet visible, or the drop is concentrated in a segment, rollback or a partial rollback is cheap relative to waiting.

Locate. Segment by platform, browser, country and new versus returning users (C13.5): if one browser accounts for most of the drop, the cause is likely a bug and the fix is quick. If the drop is spread evenly, it could be the design itself, and an experiment is the right tool.

Decide and communicate. If the drop is confirmed in the exposed group and the cause is not quickly fixable, roll back or reduce exposure to 10% while diagnosing. Tell stakeholders the five lines (what, since when, what we know, what we are doing, next update). What would change my mind: the drop disappearing when the data finishes arriving.

Follow-ups.

- **Q.** How do you tell whether the launch caused the drop? **A.** "By comparing exposed and unexposed users at the same time. If the launch was staged at 10%, conversion in the exposed 10% fell and the rest did not, the launch is the cause; if both fell, something else is happening, such as a marketing change, an outage upstream or a seasonal effect. I would also check the timing against the release hour, the mix of traffic by channel and device, and the funnel step where the drop sits. A cause outside the launch needs a different owner and a different fix."
- **Q.** When do you roll back and when do you diagnose first? **A.** "When the cost of waiting is high and the rollback is cheap and reversible, I roll back while diagnosing. If sign-ups generate \$1 million of value a day, a 3% drop costs about \$30,000 a day. My pre-set trigger would be: exposed conversion down more than 2% with an interval that excludes zero for 24 hours, so I would roll back, then diagnose in a calm environment and re-launch once the cause is fixed. If the cause is unclear and the effect is small, I would diagnose first, with a deadline."

IQ-C17-05 · Hard · execution · EX · Apple

Prompt. A hardware accessory has a software defect found two weeks before manufacturing locks. Decide.

Rubric.

- **Baseline:** "Delay and fix it," or "ship and patch."
- **Strong:** Recognizes irreversibility, sizes the defect and weighs options by cost.
- **Standout:** Tests whether the fix-later path is real (including any re-certification), uses a gate that preserves the option to delay, quantifies expected cost, considers supply and support, and treats safety as separate.

Model answer. I know the situation only in general terms; nothing here describes any real product. The first question is what kind of defect: if it can hurt someone, safety decides and it is not a tradeoff; I'll assume it is a functional defect.

Classify. The manufactured parts are irreversible after the lock; the firmware is changeable only through an update path. So the question is whether the fix-later path is real. I'd ask three things. Can the fix be delivered over the air, to units affected by the defect? Does the fix need regulatory re-certification, and how long does that take? What is the exposure window before the fix reaches customers?

Numbers (invented). The first run is 400,000 units. The defect affects 2.5% in lab tests: 10,000 units. A field problem costs about \$90 per affected unit (support, returns, replacement): \$900,000. Suppose the fix requires a three-week re-certification before it can be released. In the first three weeks, 40% of volume sells, 160,000 units, of which 2.5% are affected: 4,000 units and \$360,000 of field cost in the window. Delaying the lock by 10 days would miss the main selling window and cost an estimated \$1,500,000.

Options. (A) Lock on time; start the re-certification now so the fix is ready within about three weeks; mitigate the defect in the meantime with a documented workaround and a support script; and run a gate: by day 7, verify that the fix works on affected units and that the update path reaches them. (B) Delay the lock 10 days: \$1,500,000 for sure. (C) Lock on time and ship as is: \$900,000 of field cost, plus the reputational cost.

Compare. If the gate passes (say probability 0.75), expected cost is the window cost, \$360,000. If it fails (0.25), I delay the lock by the remaining time at, say, \$750,000. Expected: $0.75 \times \$360,000 + 0.25 \times \$750,000 = \$270,000 + \$187,500 = \$457,500$. That beats B (\$1,500,000) and C (\$900,000). The gate preserves the option to delay while buying the one piece of evidence that decides.

Also. Check that the first units out can be flagged and updated before shipping; size support; and decide who may decide at the gate. What would change my mind: the defect turning out to be safety-related, or the update path failing in the lab on day 2, in which case delay immediately.

Company note. Apple is the setting only: Inferred from third-party reports that hardware prompts at Apple include defects found near manufacturing deadlines [unverified: candidate-reported]. No claim is made about any real product, process or numbers.

Follow-ups.

- **Q.** How do you test whether fix-later is real for a hardware accessory? **A.** "By checking every link in the path. Can units receive updates (do they connect, and how often), what share of owners install an update within 30 days, does a firmware change need re-certification or regulatory notice before release [V: FCC equipment authorization, 47 CFR Part 2 and section 2.1043 on firmware (permissive changes); US rules for radio devices only; safety approvals and non-US regimes not checked], and can the fix be delivered without a visit or a return? I would test the update on a batch of 100 units, including the worst conditions, and ask support how many owners never update. If the path is unreliable for a large share of owners, the defect effectively ships in the product and the lock date matters."
- **Q.** How would you quantify the decision and handle the option to delay? **A.** "Compare expected costs. Suppose 500,000 units ship and the defect affects 3%, 15,000 units; if 30% of those are returned at \$80 each and each affected owner contacts support at \$10,

the cost is about $15,000 \times (0.3 \times \$80 + \$10) = \$510,000$, plus reputation. A two-week delay might cost \$300,000 in lost sales and supply commitments. If fix-later is reliable and the fix will reach most affected owners, shipping is cheaper; if not, delay. I would keep the option to delay open until the lock date, with a gate that reconfirms the fix path, and treat any defect that can hurt someone as outside this tradeoff."

IQ-C17-06 · Hard · execution · EX

Prompt. Your launch date is fixed and engineering says the remaining work will take three times longer than planned. Five other teams must deliver pieces. What do you do?

Rubric.

- **Baseline:** "Work harder," "add people," or "push the date."
- **Strong:** Looks at scope, dependencies and the critical path, and negotiates scope.
- **Standout:** Computes compounding dependency risk, orders scope cuts in advance, preserves the launch criteria, proposes a staged launch on the date, and communicates early with options.

Model answer. With a fixed date and a threefold gap, the lever is scope or exposure, not effort. I'd do four things this week.

Understand the gap. Ask engineering for the range and the riskiest part (C16.4), and check against the history of similar projects. Separate what is truly three times longer from what can be descope.

Map the dependencies and compound them. Five other teams, each, say, 90% likely to deliver on time: $0.9^5 = 59\%$. Even before the threefold estimate, the plan had a 41% chance of slipping on dependencies. I'd find the critical path, raise the weakest items (build against mocks, agree fallbacks, book reviews now), and cut the count on the path.

Decide the order of cuts, in advance. First polish and nice-to-haves; then secondary markets and features; never the launch criteria (quality bar, safety, accessibility, privacy). The smallest launch that delivers the core of the user's job, perhaps a third of the scope, becomes the plan for the date. The rest follows in a second release with a date.

Use staging. Launch on the date to a smaller group or one market, with the remaining scope behind flags to be enabled as it is ready. That meets the date in a meaningful way without exposing everyone to unfinished work.

Communicate. Go to the sponsor now with options and costs, not on the eve of the date: "At full scope we will miss by about X weeks; for the date we can ship the core to one market; adding people helps little at this stage (the arithmetic from C15.4); here is my recommendation." What I'd avoid: promising the full scope on the date, asking engineers to cut testing, or hoping. What would change my mind: a dependency owner offering to pull in their delivery.

Follow-ups.

- **Q.** How do you compute the dependency risk? **A.** "By multiplying probabilities. If each of five other teams has a 90% chance of delivering on time, all five do so with probability

0.9 to the fifth power, about 59%. If my own team's chance is 80% given the threefold gap, the whole launch is on time with about 47% probability. That number is more persuasive in a meeting than a statement that the plan is risky: it says that missing the date is more likely than hitting it, and it shows that adding a sixth dependency would lower it further."

- **Q.** What scope cuts and staged launch would you propose? **A.** "A cut list ordered in advance, from least to most valuable to the launch goal, agreed with stakeholders before pressure builds. On the date, launch the core to a limited group, for example 10% of users or one region, with the cut features marked as coming, and expand as the rest lands. I would keep the launch criteria (no open S1, rollback rehearsed) and say plainly what is and is not in the launch. Tell stakeholders now, with options, not a week before the date."

IQ-C17-07 · Hard · execution · EX

Prompt. It is 2 a.m. You are the PM on call. An alert shows that 5% of users cannot complete a core action since last evening's release. What do you do?

Rubric.

- **Baseline:** "Wake up the engineers and fix it."
- **Strong:** Confirms impact, contains by rollback, communicates, then diagnoses.
- **Standout:** Uses pre-agreed triggers and decision rights, runs the PM's parallel tracks (impact, communication, decisions, log), weighs rollback against fix forward, and sets up a blameless postmortem.

Model answer. My job is not to debug; it is to make sure the right people are working the problem, the impact is limited, and everyone who needs to know does.

First 15 minutes (FW-C17-06). (1) *Confirm impact:* who, how many, since when, how severe? Is the alert real (a second source)? Five percent of users failing at a core action is severe. (2) *Contain:* the release was last evening; if it was behind a flag, the on-call engineer can turn it off, and the trigger sheet probably says to. I'd confirm that and not debate. If it cannot be flagged, decide quickly between a rollback and a fix forward: rollback is usually safer if the previous version is known good and the rollback is rehearsed. (3) *Communicate:* an internal status in the incident channel with the five lines, and a plan for the customer message and the status page if it will last past the morning. Tell support what to say. (4) *Decide the tradeoffs the engineers bring:* for example, disabling the new feature entirely for 100% of users or only for the affected 5%; the PM is the person who weighs customer impact and scope. (5) *Log:* keep a timeline.

Protect the team. Provide one voice upward with a status every 30 to 60 minutes, so the engineers working the fix are not interrupted by questions.

After containment. Diagnose with the C13 sequence (what changed, who is affected, timing, location, size), confirm that metrics recover, and decide whether to relaunch with a fix and a smaller ramp.

Postmortem. Within a few days, a blameless review: timeline, root causes, why testing and monitoring did not catch it earlier, what went well, and actions with owners and dates (for example, an alert on the core action's success rate, a canary stage). I'd own the product-impact and decision sections and follow up the actions.

What would change my mind: if the rollback itself is risky (a data change), in which case a targeted disable and fix forward may be better, and I'd say so with the reasoning.

Follow-ups.

- **Q.** Rollback or fix forward? **A.** "Roll back when a flag or a rehearsed rollback exists and the cause is not understood, because restoring service quickly limits the harm. Fix forward only if the fix is small, understood, tested and will land faster than the rollback. The test is time to restore service for the affected 5% of users, not how elegant the fix is. If the rollback would lose data written since the release, I would say that and let the incident commander decide with engineering. I would not debug myself; my job is to make sure the decision is made by the right person quickly."
- **Q.** What do you do in the postmortem? **A.** "Run it blameless, within a few days. Write the timeline, the contributing causes and what made the problem late to detect, since 5% of users failing for hours means the alert or the test was missing. Agree action items with owners and dates, such as a canary stage, a guardrail alert on the core action and a rollback drill, and check whether the release rule was followed. Share the writeup with the team so that others learn, and review the action items at the next release review to see that they were done."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Figures are assumptions unless marked, and the arithmetic was computed.

EX-C17-01 · Size the stages

Problem. A product with 2,000,000 users has an error rate of 2%. Using the figures that the stage sizes needed (comparing the exposed group with all other users, 80% power, 5% level) are about 7,700 users to detect a rise of 0.5 points, 2,300 to detect 1.0 point and 43,100 to detect 0.2 points, say what a 1% canary (20,000 users) and a 10% stage (200,000 users) can detect, and propose stage sizes for a worry of "any rise over 0.2 points."

Context. A rollout plan is being written.

Data. As above.

Constraints. Five minutes.

Expected approach. FW-C17-02 and Worked example 1.

Solution. A 1% canary of 20,000 users is more than the 7,700 needed for 0.5 points and the 2,300 for 1.0 point, so it reliably sees a rise of about 0.3 points or more (between the 0.5-point and 0.2-point figures, closer to 0.3). To detect a rise as small as 0.2 points, 43,100

users are needed, so the 1% canary is not enough and the 10% stage (200,000) is: it can see even smaller rises, about 0.1 points. Plan: canary 1% for severe failures; 10% (or at least 3%, about 60,000 users, which exceeds 43,100) for the 0.2-point worry; dwell a business day at each. State that harms below 0.1 points will not be visible at these sizes and are watched through lagging indicators.

Alternative solution. Add a 3% stage between 1% and 10% to keep the blast radius small while reaching the 0.2-point size.

Tradeoffs. More stages cost days; fewer stages widen the blast radius.

EX-C17-02 · Set triggers with noise in mind

Problem. A service has a success rate of 95% and about 1,000 attempts per half hour. Compute the standard error over 30 minutes and over 2 hours (4,000 attempts). Propose a threshold at about 3 standard errors for each window, and the chance of a false alarm per window at 2 standard errors and at 3.

Context. A rollback trigger sheet.

Data. As above.

Constraints. Six minutes.

Expected approach. FW-C17-05 and Worked example 4.

Solution. Thirty minutes: $\sqrt{(0.95 \times 0.05 \div 1,000)} = 0.69$ points. Two hours: $\sqrt{(0.95 \times 0.05 \div 4,000)} = 0.34$ points. Thresholds at about 3 standard errors: 30 minutes, a drop of about 2.1 points (to about 92.9%); 2 hours, a drop of about 1.0 point (to about 94.0%). The chance of a false alarm per window by luck is about 2.3% at 2 standard errors and 0.13% at 3 (one-sided). With 48 half-hour windows a day, a 2-standard-error trigger would fire falsely about once a day ($48 \times 0.023 = 1.1$); a 3-standard-error trigger about once in 16 days ($48 \times 0.0013 = 0.06$). Use the 3-standard-error thresholds, and a coarse fast trigger (for example below 90% in 15 minutes) for large failures.

Alternative solution. Use a sequential test that accumulates evidence, so large drops fire fast and small drops fire slowly (a data scientist can set it up).

Tradeoffs. Tight triggers cause false rollbacks and teach people to ignore alerts; loose ones fire late.

EX-C17-03 · Compound the dependencies

Problem. Six independent dependencies are each 92% likely to be on time. Compute the chance all arrive on time. If you raise two to 98% by early engagement, what is the new chance? Give the order of three scope cuts.

Context. A launch plan.

Data. As above.

Constraints. Five minutes.

Expected approach. FW-C17-07.

Solution. $0.92^6 = 0.61$, so a 39% chance that something slips. With two raised to 98%: $0.92^4 \times 0.98^2 = 0.7164 \times 0.9604 = 0.688$. So about 69%. The remaining risk justifies a buffer and a cut order: (1) placeholders instead of final design assets, (2) fewer markets at launch, (3) a smaller first feature set. Launch criteria are never cut.

Alternative solution. Reduce the number of dependencies on the critical path from six to four by launching in one market, which at 92% each gives $0.92^4 = 72\%$ even without raising any.

Tradeoffs. Early engagement costs time now and buys probability later.

EX-C17-04 · Classify reversibility

Problem. Classify as reversible, hard to reverse or irreversible, and say what you would change in the launch approach: (a) a flagged recommendation tweak; (b) a price increase announced to customers; (c) a hardware part after the tooling lock; (d) a database migration that drops old columns; (e) an email sent to 2 million customers.

Context. Launch planning.

Data. None.

Constraints. Six minutes.

Expected approach. FW-C17-03.

Solution. (a) Reversible: flag, stage, roll back on triggers. (b) Hard to reverse: reversing a price after announcing it costs trust; get evidence first, stage by market, prepare the retraction. (c) Irreversible: the highest bar; gate the lock on tests; validate the fix-later path; consider safety. (d) Irreversible without a backup: take a full backup, migrate in steps (add new, copy, switch, drop later), and rehearse. (e) Irreversible once sent: test with small seed groups, proofread, schedule with a pause window and a send-to-staff first.

Alternative solution. Treat (b) as reversible-with-cost and compute the cost of reversal.

Tradeoffs. Each added check has a cost; spend more where the door cannot be reopened.

EX-C17-05 · Launch or hold

Problem. A test shows purchases +0.5 points [+0.2, +0.8] against a threshold of +0.4, an error-rate guardrail +0.1 points [−0.1, +0.3] against a margin of +0.2, and no validity issues. The change is behind a flag. What do you do?

Context. The rule was fixed beforehand (strict version).

Data. As above.

Constraints. Six minutes.

Expected approach. FW-C17-04.

Solution. The primary's lower end (+0.2) is below the threshold (+0.4), so the primary is not yet cleared by the strict rule. The error guardrail's upper end (+0.3) exceeds the margin (+0.2): a possible breach. Under the strict rule: do not launch as is. But the change is

reversible and flagged, the estimate is above the threshold and the point estimate of the error change is within the margin. A sensible action: investigate the errors (what are they?), extend the test at a larger exposure (a staged launch with a holdout and the error trigger as the rollback), and decide when the intervals narrow. Four times the data halves the width.

Alternative solution. If the errors are cosmetic and cheap, fix and retest quickly.

Tradeoffs. Waiting costs the gain; launching fully risks a harm that has not been ruled out. The staged path gets both information and some of the gain.

EX-C17-06 · Build a pre-launch checklist

Problem. Write the pre-launch checklist for a feature that lets users send money to friends. Name at least two items in each of the eight categories.

Context. A payments feature with legal and fraud implications.

Data. None.

Constraints. Eight minutes.

Expected approach. C17.7 and FW-C17-01.

Solution. *Product:* amounts and fees displayed correctly, no open S1/S2 on the send path; accessibility pass. *Reliability:* tested at 3× expected volume, idempotent sends (no double charges), rollback and kill switch rehearsed. *Data:* events for each step; a dashboard of success and failure rates; baseline fraud rate. *Rollout:* ladder with dollar limits per user at each stage; triggers on failed sends, wrong amounts and fraud. *People:* support scripts for "money not received," dispute process, trained team. *Constraints:* legal and compliance sign-off on money transmission rules, fraud and risk review, privacy review, partner bank readiness. *Accessibility and localization:* screen reader labels on amounts; currency formats; supported languages. *Decision:* go or no-go with the risk owner; dissent recorded.

Alternative solution. Add a limited pilot with employees sending small amounts.

Tradeoffs. A long list is useless if not owned; give each item an owner and a date.

EX-C17-07 · The lock-date decision, with numbers

Problem. A first run is 300,000 units. A firmware defect affects 3% of units in lab tests; each affected unit costs \$80 in the field. Delaying the lock 10 days costs \$1,200,000. A gate on day 7 tests whether an update fixes the defect: it passes with probability 0.7, leaving 15% of affected units unfixed (cost then \$108,000); if it fails, you delay the lock by 5 days at a cost of \$600,000. Compute the expected cost of each option: gate, delay now, ship as is.

Context. A hardware decision.

Data. As above.

Constraints. Seven minutes.

Expected approach. Worked example 3.

Solution. Affected units: $3\% \times 300,000 = 9,000$; shipping as is costs $9,000 \times \$80 = \$720,000$. Delay now: $\$1,200,000$. Gate: $0.7 \times \$108,000 + 0.3 \times \$600,000 = \$75,600 + \$180,000 = \$255,600$. The gate is cheapest. Sensitivity: if the pass probability were 0.4, the gate costs $0.4 \times \$108,000 + 0.6 \times \$600,000 = \$43,200 + \$360,000 = \$403,200$, still below shipping as is. With these numbers the gate beats shipping as is for any pass probability, because even its worst case (a 5-day delay at $\$600,000$) costs less than shipping the defect ($\$720,000$). It would lose only if a failed gate forced the full 10-day delay ($\$1,200,000$) and the pass chance were low.

Alternative solution. Add a safety check: if any affected unit can overheat or injure, the calculus is not cost.

Tradeoffs. The gate buys information; the cost is the effort of running it fast.

EX-C17-08 · Write a postmortem outline

Problem. An outage after a release left 5% of users unable to log in for 4 hours. Write the outline of a blameless postmortem, with two example action items.

Context. You co-own it as PM.

Data. Invent plausible details and mark them (invented).

Constraints. Seven minutes.

Expected approach. C17.5.

Solution. *Summary:* 5% of users could not log in for 4 hours; no data lost (invented). *Impact:* about 50,000 users, 1,200 support contacts, a failed partner demo. *Timeline:* release at 21:00; first support contacts 21:40; alert fired at 22:30 (late); rollback decision 23:00; rollback complete at 01:00. *Root causes:* a configuration difference between the test and production authentication service; no canary stage; the login failure rate was not alerting. *What went well:* clear incident roles; communications every 30 minutes. *What went poorly:* detection took 90 minutes; rollback took 2 hours because it had not been rehearsed. *Actions (owner, date):* add an alert on login success by version (engineering, one week); add a 1% canary stage to the release ladder (release manager, two weeks); rehearse the rollback (engineering and PM, one week). *Language:* about the system, not about people.

Alternative solution. Add a "where we got lucky" section.

Tradeoffs. A postmortem costs time and returns it in fewer repeats; actions must be owned and checked.

EX-C17-09 · Plan the post-launch review

Problem. A new onboarding flow launches. Define the day-1, week-1 and week-4 checks, the expected values, and the decision to iterate, scale or stop.

Context. Success criteria must be written before launch.

Data. Invent targets and mark them (invented).

Constraints. Seven minutes.

Expected approach. FW-C17-08.

Solution. *Day 1:* health: errors under 1%, onboarding completion not below baseline minus 2 points, support contacts normal, no trigger hit. *Week 1:* activation (first project created within 24 hours) target 40% against a baseline of 33% (invented); guardrails: sign-up completion, uninstalls; read it with the novelty caution, comparing new users with the holdout. *Week 4:* retention at day 28 not below the holdout; the long-term holdout (2%) compared; qualitative feedback. *Decision rules:* iterate if activation is up at least 3 points and a guardrail is borderline; scale if up at least 5 points with guardrails clear; stop if up less than 1 point at week 4 or any guardrail breaches. Write the launch review with what surprised the team.

Alternative solution. Add a week-12 check for retention if the product's cycle is long.

Tradeoffs. More checkpoints cost attention; set them where decisions are made.

EX-C17-10 · A timed full answer

Problem. Give a 25-minute answer to: "You are launching a new 'live location sharing' feature for a messaging app in six weeks. Walk me through it."

Context. Self-scored; record yourself. State assumptions aloud.

Data. None.

Constraints. Use the 25-minute budget in section 4.

Expected approach. The full structure, scored against the six behaviors in section 1.

Solution. An outline of a strong answer. Clarify: the feature shows your location to chosen contacts for a set time; privacy and safety make the stakes high. Readiness gate: privacy and safety review (misuse by stalkers or abusers: default off, visible indicator, time limits, easy stop), legal for location data rules, accessibility, support scripts for harassment reports, instrumentation. Rollout ladder: staff, 1% with limited durations, 10%, 50%, 100%, sized to detect a rise in harassment reports (a guardrail). Triggers: abuse reports above a threshold per 10,000 shares; location accuracy complaints; battery drain.

Mixed-result rule: any safety guardrail breach pauses the ramp regardless of engagement. Dependencies: map provider, legal, safety team, localization; compound them; scope cuts: fewer durations, one market first. Measurement: shares per user, repeat use, safety guardrails, day 1/week 1/week 4. Irreversibility: sharing location cannot be un-shared after the fact; so the defaults and the stop controls matter more than usual.

Alternative solution. Start with a smaller feature (arrival notifications) to learn before the live feature.

Tradeoffs. A feature with safety risks needs a higher bar than engagement metrics suggest.

13. Practice protocol

Timed drills (three a week for three weeks).

- *Week 1: readiness and rollout.* Use IQ-C17-01, IQ-C17-02, EX-C17-01 and EX-C17-06. Write a readiness gate, size the stages with arithmetic and say the ladder aloud.
- *Week 2: triggers, decisions and risk.* Use IQ-C17-03, IQ-C17-04, IQ-C17-06, EX-C17-02, EX-C17-03 and EX-C17-05. Compute the noise for triggers, compound dependencies, and decide launch or hold with a rule.
- *Week 3: irreversibility, incidents and the full answer.* Use IQ-C17-05, IQ-C17-07, EX-C17-04, EX-C17-07, EX-C17-08 and EX-C17-10. Run an incident drill with a partner who changes the facts, and a full 25-minute launch answer.

With a partner. Ask your partner to say "Just ship it, we'll fix it later" and "Why 1%?" at random, and to reveal a surprise midway (a defect, a dependency slip). Practice returning to criteria and arithmetic.

Rehearse a rollback. If you can, watch a real rollback or a game-day exercise on your team, and time it.

Feedback checklist (yes, partly, no).

1. Did I define readiness with criteria, owners and a named decider?
2. Did I stage the rollout and size stages to the harm I fear?
3. Did I set rollback triggers with thresholds and windows that respect noise?
4. Did I use a pre-set rule for mixed results and offer partial options?
5. Did I classify reversibility and, for irreversible steps, test the fix-later path?
6. Did I compound dependency risk and order scope cuts?
7. Did I describe the PM's role in an incident (confirm, contain, communicate, decide, log)?
8. Did I plan the post-launch review against criteria written beforehand?
9. Did I treat safety as outside the tradeoff?
10. Did I finish with the two-sentence close from section 4?

Answer journal. For each drill record the date, the prompt, the gate, the ladder, the triggers, your score out of ten and the step you rushed.

14. Implementation playbook

Apply the chapter on real work over the next three weeks. Each deliverable doubles as an interview story.

Days 1 to 3. Take your next launch (or the last one) and write the readiness gate (FW-C17-01): criteria in each category, an owner for each and a named decider. Review it with your engineering lead and ask what is missing.

Week 1. Draft the rollout ladder and the trigger sheet for the launch. Work out what each stage can detect (use the arithmetic of Worked example 1) and what the noise is for each trigger metric (C13). Agree the sheet with the on-call engineer and support.

Week 1. Check that the rollback or off-switch exists and has been tried. If it has not, schedule a rehearsal before the launch.

Week 2. Map the dependencies (FW-C17-07), compute the compounded probability, and book the longest-lead items. Write the order of scope cuts and share it with your sponsor.

Week 2. Write down the rule for mixed experiment results for this launch (FW-C17-04) before the data arrive.

Week 3. Book the day-1, week-1 and week-4 reviews, with the success criteria and the decision rules (iterate, scale, stop). After launch, write the launch review, including what surprised you.

Deliverables that become stories: the readiness gate and trigger sheet (a time you prepared for failure and it paid off), the dependency map (a time you turned vague risk into a plan), and the launch review (a time you learned from a launch and shared it). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. Launch readiness is a set of criteria with owners and a named go or no-go decider, written early and waived only in writing; the date is not the goal.
2. Limit the blast radius with flags and a staged ladder, size each stage to the smallest harm you will not accept, and set rollback triggers beforehand with thresholds and windows that respect noise.
3. For mixed experiment results, use a rule fixed before the data and prefer partial options (staged launch with a holdout, extension, fix and retest) to an all-or-nothing choice.
4. Irreversible steps (hardware, public commitments, data deletion) need a higher bar: gate the lock on tests, validate the fix-later path, quantify delay against field cost, and treat safety as outside the tradeoff.
5. Dependency risk compounds, so map it, raise and reduce it, and order scope cuts in advance; in an incident, the PM confirms impact, contains, communicates, decides tradeoffs and logs, then leads a blameless postmortem; and a launch is followed by reviews at day one, week one and week four.

Further reading

- Betsy Beyer and others (eds.), *Site Reliability Engineering* (2016), chapters on release engineering, incident response and postmortem culture [V: Site Reliability Engineering, O'Reilly, 2016].

- Ron Kohavi, Diane Tang and Ya Xu, *Trustworthy Online Controlled Experiments* (2020). Ramping, guardrails and monitoring.
- James Reason, *Human Error* (Cambridge University Press, 1990) and *Managing the Risks of Organizational Accidents* (Ashgate, 1997), on layered defenses; the Swiss cheese model is set out in full in the 1997 book [V: Human Error: Cambridge University Press, 1990; Managing the Risks of Organizational Accidents: Ashgate, 1997, where the Swiss cheese model is developed].
- Diane Vaughan, *The Challenger Launch Decision* (1996), on the normalization of deviance [V: University of Chicago Press, 1996 (hardback ISBN 9780226851754; paperback 9780226851761); a 2016 edition adds a new preface].
- Meta's engineering account of the October 2021 outage [V: Meta engineering post of 5 October 2021], and cloud providers' public post-incident reviews, as examples of how incidents are described.
- Samsung's Newsroom page on its investigation announcement of 23 January 2017, AP and Reuters reports of the recall and the investigation (September 2016 to January 2017), and trade-press reports of the December 2016 software update. The sources for CS-C17-02.
- Frederick P. Brooks, Jr., *The Mythical Man-Month* (anniversary edition, 1995), on schedules and staffing [V: Brooks, 1975 with editions 1982 and 1995].